

DURATION: 20 MINUTES

FULL MARKS: 15(+2)

ID:

1. Consider a K-Nearest Neighbor classifier with N training examples. Suppose we set $K = N$. Assume there is no tie in the majority vote. 4

How will its predicted label for a test input differ from the prediction of a 1-NN classifier on the same training set? Explain what this implies for the decision regions.

Solution:

Expected Answer: With $K = N$, the classifier, the class that has the majority in the training set will always win the majority vote, so any test sample, irrespective of value, will always be classified as the majority training class. The decision boundary will also just be one class (ie. no boundary at all).

Marking Rubrics (4 marks):

- Full (4) marks for any mention of "always predict one class"
- Up-to 2 marks for any valid comparison with 1-NNs decision boundary.

2. A linear image classifier assigns class scores 5

$$s_j(x) = w_j^T x + b_j$$

for class j , where x is the vectorized image. At test time, every pixel is shifted by the same value c , so the classifier receives

$$x' = x + c.$$

Will the predicted class necessarily remain unchanged? Justify your answer using the score function.

Solution:

Expected Answer: The prediction does not necessarily remain unchanged. Under the shift,

$$s_j(x') = w_j^T(x + c) + b_j = s_j(x) + c \sum_k w_{j,k}.$$

Each class score shifts by an amount depending on the sum of that class template's weights. If these shifts differ across classes, the ordering of scores can change, so the predicted class can change. The class with the highest sum-of-W-entries will have the highest shift.

(Additional discussion about invariance, not necessary for marks): A sufficient condition for invariance is

$$\sum_k w_{j,k} = \alpha \quad \text{for all classes } j.$$

Then every score receives the same additive offset $c\alpha$, so the argmax is unchanged. For a particular image, it is also enough that the original winning class still has the highest shifted score.

Marking Rubrics (5 marks):

- 2 marks: Writing any variant of the word "no"
- Up-to 3 marks based on explanation of amount of score change, with full 3 marks being awarded if discussion of sum-of-W-entries is present.

3. Regular SGD observes the following gradient sequence for a two-parameter model:

$$g_1 = (100, 1), \quad g_2 = (-100, 1), \quad g_3 = (100, 1), \quad g_4 = (-100, 1), \dots$$

The first coordinate has large magnitude and changes sign because the loss changes sharply in that direction. The second coordinate has small magnitude but is consistent because the loss changes slowly in that direction.

Compare how the behavior would change if we replaced regular SGD with the following optimizers:

$$\begin{aligned} \text{Momentum: } v_t &= \rho v_{t-1} + g_t, & w_{t+1} &= w_t - \alpha v_t \\ \text{AdaGrad: } G_t &= G_{t-1} + g_t^2, & w_{t+1} &= w_t - \alpha \frac{g_t}{\sqrt{G_t} + \epsilon} \end{aligned}$$

- SGD with Momentum:
- AdaGrad:

Solution:

Expected Answer: SGD with Momentum averages gradients through the velocity. The large alternating first-coordinate gradients partially cancel, while the small consistent second-coordinate gradients accumulate. It should reduce zig-zagging to near-zero and move more smoothly in the consistent direction.

AdaGrad accumulates g_t^2 per coordinate. The first coordinate receives very large squared gradients, so its effective learning rate is reduced strongly. The second coordinate has small squared gradients, so it is damped less. It should reduce oscillation in the sharp direction while preserving the same progress in the slow direction.

Both optimizers will behave somewhat similarly in this case, but Momentum will reduce zig-zagging in the first-dimension faster and more drastically compared to AdaGrad.

Marking Rubrics (6 marks):

- Up-to 3 marks: Correct momentum explanation.
- Up-to 3 marks: Correct AdaGrad explanation.

4. **(Bonus Question)** AdaGrad uses the following update:

$$\begin{aligned} G_t &= G_{t-1} + g_t^2, \\ w_{t+1} &= w_t - \alpha \frac{g_t}{\sqrt{G_t} + \epsilon}. \end{aligned}$$

Suppose a student changes the accumulator to $G_t = G_{t-1} + g_t$. How do you expect the behavior of the optimizer to change?

Solution:

Expected Answer: The accumulator no longer stores a nonnegative estimate of squared gradient magnitude. Since signed gradients can cancel, a coordinate with large oscillating gradients may appear small (e.g., in the case of the previous question), so the optimizer may fail to reduce the learning rate in that coordinate. The accumulator can also become negative, making $\sqrt{G_t}$ undefined or unstable. This will completely break AdaGrad's adaptive learning-rate behavior.

Marking Rubrics (2 marks):

- Any valid explanation will warrant 2 marks, even if it does not match this expected answer.